오픈소스 반도체 설계 및 제작 종합설계 #6

AI 고위합성 결과의 검증과 내 칩 설계 계획

2021-17385
산업공학과 김예원

[1] HLS 예제와 AI 대화 검토

이번 주에는 고위합성(HLS, High Level Synthesis) 예제 두 가지를 검토하였다. 고위합성은 C/C++로 작성한 알고리즘을 베릴로그 RTL 로 바꾸는 과정이며, 이번 예제에서는 이 변환을 AI(Gemini)가 담당했다. 각 예제에는 C++ 원본(c_untimed), AI 와의 대화 기록(Talks), AI 가 만든 RTL, 시뮬레이션과 에뮬레이션 환경이 들어 있다.

첫 번째 예제인 시계(Clock_with_Gemini)는 함수를 부를 때마다 1 초를 더하는 C++ 코드를 RTL 로 바꾼 것이다. AI 는 함수 호출을 클럭 한 번으로, 함수 안의 static 변수를 레지스터로 대응시켰고, 시·분·초의 비트 폭을 각각 5·6·6 비트로 줄였다. 다만 계산용 레지스터 뒤에 출력용 레지스터를 한 단계 더 두어, C++과 달리 새 값이 한 클럭 늦게 출력되는 구조가 되었다.

두 번째 예제인 8 탭 FIR 필터(fir8_hls_with_Gemini)는 최근 입력 8 개에 계수 {4, 12, 25, 34, 34, 25, 12, 4}를 곱해 더하는 디지털 필터이다. 대화는 두 단계로 진행되었다. 먼저 C++ 코드의 출력을 8 비트로 줄이는 과정에서 AI 는 255 를 넘는 값을 255 로 자르는 포화 방식을 제안했으나, 계수 합이 150 이라 대부분의 출력이 255 로 잘리는 문제를 지적하였고, AI 는 128 로 나누는 방식(오른쪽 7 비트 시프트)으로 바꾸었다. 이어서 RTL 을 만들면서 하드웨어를 계속 줄이도록 요구하자, AI 는 병렬, 직렬, RAM 사용, 비트-시리얼, 플립플롭과 고정 배선의 다섯 가지 구조를 차례로 제시하며 각각의 레이턴시와 면적 절감을 주장하였다.

[2] AI 가 만든 결과의 검증

복습 글에서 AI 의 결과는 인상적이지만 완벽하지 않으므로 C++ 원본과 비교해 확인해야 한다고 하였다. 이에 따라 AI 가 대화에서 주장한 내용 네 가지를 실행으로 확인하였다.

첫째, AI 는 원본 C++ 반복문에 '치명적인 버그'가 있다며 코드를 고쳤다. 무작위 입력 10 만 개를 원본과 AI 수정본, FIR 정의식에 각각 넣어 비교한 결과 세 결과가 모두 일치하였다. 원본 반복문은 뒤에서부터 돌며 값을 덮어쓰기 전에 읽기 때문에 처음부터 올바른 계산이었고, AI 는 없는 버그를 고쳤다고 설명한 것이었다.

둘째, 128 로 나누는 방식은 계수 합 150 과 맞지 않아 이득이 약 1.17 배가 된다. 같은 값을 계속 입력하는 경우를 계산해 보니 입력 219 부터 결과가 255 를 넘었고, 포화 처리가 없으면 8 비트로 넘쳐 입력 255 에서 출력이 43 이 되는 등 큰 값이 작은 값으로 뒤집혔다. 교수님의 C++에 포화 옵션(FIX_SATURATION)이 추가되어 있고 RTL 에 255 제한이 들어 있는 이유를 확인하였다.

셋째와 넷째로, 다섯 가지 RTL 이 정확히 동작하는지와 실제 크기를 확인하였다. 각 RTL 을 Verilator 로 시뮬레이션하며 테스트벤치와 같은 방식(입력 하나를 넣고 출력 유효 신호를 기다림)으로 입력 300 개를 넣어

C++ 기준값과 비교하였고, 입력 간격을 1~80 클럭으로 바꾸며 정상 동작하는 최소 간격을 찾았다. 크기는 ETRI 0.5µm 표준 셀로 합성하여 게이트 수로 환산하였다(표 1).

표 1. AI 가 만든 FIR RTL 다섯 버전의 검증 결과

버전	AI 의 주장	실제 동작 (입력 300 개)	합성 크기
0 병렬	곱셈 8 개 동시	300/300 정확, 결과는 3 샘플 늦게 출력	약 2,640 GE
1 직렬	곱셈기 1 개, 9 클럭	16%만 정확, 합성 불가	-
2 RAM	면적 최소	6% 이하 정확, 출력 유효 신호 오류	약 1,880 GE
3 비트-시리얼	곱셈기 0 개, 18 클럭	실제 66 클럭, 60%가 ±1 오차	약 1,230 GE
4 FF + 고정 배선	곱셈기 0 개, 11 클럭	문법 오류(megedge 오타). 수정 시 300/300, 실제 13 클럭	약 2,430 GE

결과적으로 그대로 사용할 수 있는 버전은 병렬 구조 하나였고, 오타를 고친 4 번을 더해도 두 개였다. AI 가 면적을 줄이라는 요구에 답할수록 코드가 달라졌고, AI 가 제시한 레이턴시도 실제와 달랐다(3 번 18 → 66 클럭, 4 번 11 → 13 클럭). 또한 '곱셈기 0 개' 구조인 4 번은 병렬 구조보다 약 8%만 작았다. 계수가 상수이면 병렬 구조의 곱셈도 합성기가 시프트와 덧셈으로 바꾸기 때문이며, 이는 4 주차에 AI 의 면적 평가를 합성으로 검증했을 때와 같은 원인이다. 속도를 양보해 면적을 줄이는 효과가 뚜렷했던 것은 66 클럭을 쓰는 3 번뿐이었다. 직렬 구조에서는 입력을 얼마나 자주 넣을 수 있는지와 출력이 언제 유효한지가 정확도를 좌우하였다.

[3] 알고리즘을 하드웨어로 만드는 이유

블로그에서는 CPU 로는 어떤 한계가 있어서 알고리즘을 하드웨어로 만드는지 질문하였다. FIR 필터를 예로 들면, CPU 는 곱셈과 덧셈 8 쌍을 명령어로 차례로 실행하므로 출력 하나에 여러 번의 명령 실행이 필요하다. 반면 병렬 RTL 은 곱셈과 덧셈을 회로로 펼쳐 두어 매 클럭 새 입력을 받아 처리하였다(표 1 의 0 번, 입력 간격 1 클럭). 같은 계산을 정해진 시간 안에 반복해야 하는 신호 처리와 신경망 연산에서 이 차이가 하드웨어를 만드는 이유가 된다. 다만 그 대가로 회로 면적이 커지고, 면적을 줄이면 클럭 수가 늘어나는 맞바꿈이 생긴다. 이번 검증에서 이 맞바꿈은 AI 의 설명이 아니라 시뮬레이션과 합성 결과로만 확인할 수 있었다.

[4] 내 칩 설계 계획: 곱셈 없는 신경망 층 (Bitnet)

My chip 에 제출할 주제롤 고민해보았다. 주제를 정하면서 최근에 새롭게 들어본 BitNet 을 구현해보고 싶다는 생각이 들었다. BitNet 은 신경망의 가중치를 {+1, 0, −1}의 세 값으로 제한하는 방식이다. 신경망의 한 층은 $y = Wx + b$ 의 행렬 곱인데, 가중치가 세 값뿐이면 곱셈이 '더하기, 빼기, 건너뛰기'로 바뀌어 곱셈기가 필요 없어진다. My chip 은 RAM 블록이 없고 코어가 약 $1mm^2$ 인 작은 칩이므로, 가중치를 칩에 고정하고 곱셈기를 없애는 BitNet 방식이 제약에 맞다고 판단하였다.

1 차 목표는 2 주차에 학습한 3×8 디코더 신경망을 3 진 가중치로 바꾸어 칩으로 만드는 것이며, 진행 계획은 표 2 와 같다. 모든 단계에서 파이썬 결과를 정답으로 두고, 이번 주의 검증 방법대로 같은 입력에 같은 출력을 내는지 확인한다.

표 2. 내 칩 설계 진행 계획 (3 진 가중치 3×8 디코더)

단계	구분	할 일	확인 기준
1	알고리즘	파이썬으로 학습한 뒤 가중치를 {+1, 0, −1}로 양자화	실수 모델과 3 진 모델의 정확도 비교
2	C++ 모델	정수만 쓰는 추론 함수 (곱셈 없이 더하기·빼기)	파이썬 3 진 모델과 출력 일치
3	RTL	AI 를 이용해 C++ → 베릴로그 (병렬 구조부터)	Verilator lint 통과, 조합·순차 블록 분리
4	시뮬레이션	SystemC 테스트벤치로 C++ 모델과 비교	모든 입력에서 C++과 일치
5	합성·비교	ETRI050 합성. 병렬/직렬, 활성값 비트 수별 면적 비교	코어 안에 들어가는 크기
6	에뮬레이션	FPGA 보드에서 실행	시뮬레이션과 같은 결과
7	칩 도면	배치·배선, LVS·DRC, 적층 비아 검사, 패드 부착, GDS	LVS 일치, DRC 오류 0